Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

Keep this section brief, typically 2 to 4 sentences.
Focus on:

- A significant decision I made was to add a debug HUD overlay that could be toggled on and off, since I wanted something that felt practical for actual playtesting rather than just fulfilling the requirement
- I also decided to include god mode as my second toggle feature since it would be the most useful when testing level layouts and enemy placement without constantly dying.

Examples:

- T = HUD interface toggle on/off
- M = moon gravity mode on/off
- G = God mode on/off

Role-Based Process Evidence

Entry Header

Name: Katherine Chen
Role(s): Main Coder/Editor/AI prompter
Primary responsibility for this work: Using AI to implement Week 9's debug screen requirements, as well as being the human decision point on what debug features would actually be useful for game dev irl.

Goal of Work Session

Examples:

- Adding a toggleable debug HUD overlay that displays live player and world stats
- Implementing moon gravity and god mode as toggleable debug features accessible via keyboard shortcuts

Tools, Resources, or Inputs Used

- GenAI:
    - Claude Sonnett 4.6
- VSCode
- GitHub

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 24th, 2026

- **Tool Disclosure**: Claude Sonnett 4.6

Purpose of Use: I had a clear idea of what I wanted the debug screen to look like and what features it should have, but with how many interconnected files this project has I used AI to

figure out exactly which files needed to be changed and implement my vision across all of them efficiently.

Summary of Interaction: The tool was mainly in charge of implementing my ideas across the correct files. I told it what features I wanted (HUD overlay, moon gravity, god mode) and it wrote the changes into the existing code structure. It also helped me troubleshoot why the M and G keys weren't working, which turned out to be because Game.js was never calling the toggle functions, ect.

Human Decision Point(s): The decision of what the debug screen should look like, what stats to display, and which features to include all came from me. I decided on god mode as the second toggle because it made the most practical sense for playtesting. I also personally went into the code and verified the changes made sense given how the existing systems were set up, since I understood the codebase well enough to catch when something was being written incorrectly.

Integrity & Verification Note: Since I personally know HTML, JavaScript, and CSS, I was able to verify most of what the AI wrote and understood that it was mostly correct. I also playtested repeatedly on live server+github to confirm the keys were working, the HUD was displaying accurate values, and that moon gravity properly reset on restart.

Scope of GenAI Use: Like mentioned earlier, GenAI did not contribute to the creative or design portions of this. The decision of what features to debug, how the panel should look, and what information was actually useful to a developer all came from me.

Limitations or Misfires: The tool initially failed to implement moon gravity once and god mod twice. I had to feed it the additional files so it could trace the full data flow and fix it properly. It also initially mapped the debug toggle to D instead of keeping it on T, which I had to correct.

Summary of Process (Human + Tool)

- I came up with my own vision for what the debug screen should show and which features to toggle
- I used AI to implement those ideas across the correct files in the existing codebase
- I used it to troubleshoot why M, G, and moon gravity reset were not working
- I did my own verification by playtesting and cross referencing the code with my understanding of context

Decision Points & Trade-offs

- I chose god mode as the second toggle over something like slow mo because it was the most immediately useful for playtesting enemy encounters (esp if im a bad gamer and just want to get through stuff fast)
- I kept the debug key as T instead of switching it to D since the original InputManager already had T mapped and changing it would have been unnecessary

Verification & Judgement

- Playtesting on live server was a big part of making sure everything worked. I tested each key individually, confirmed everything worked and nothing broke due to my changes
- I also cross referenced the changes with my understanding of how the general code architecture works to make sure nothing was being wired in the wrong layer

Limitations, Dead Ends, or Open Questions

- The god mode wasn't working at first because the fix needed changes across three files and AI didn't realize that until I uploaded them all separately
- I wanted to add a hitbox toggle but couldn't figure out how p5play handles collider drawing in time so I left it for later
- GenAI needs the full context to make all the changes at once but Claude only lets me upload so many files so I had to debug a few times and upload files as I debugged

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness. **Note**: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

Prompt	**Answer/Generation**
Here are the attatchments of the main file let me know if you need other files. I need to add 2 debugging features: - Key M: Moon gravity mode: User is able to jump very high with nearly zero gravity, surpassing many platforms and levels - Key G: God mode, user is able to take no damage whatsoever Both these modes must show on or off when you open T HUB overaly interface	No problem! So your debug screen will be: • **T key** — toggle the overlay on/off • **M key** — toggle moon gravity • **G key** — toggle god mode To give you the exact code, I need your src files. Can you upload these three: 1. `src/DebugOverlay.js` 2. `src/InputManager.js` 3. `src/world/PlayerController.js`
[upload files] here you go	I have all three files. Here are the exact changes for each file: `src/DebugOverlay.js` — full rewrite Replace the entire file with this:

Rewritten DebugOverlay with HUD panel, moon gravity, god mode toggles
Rewritten DebugOverlay with HUD panel, moon gravity, god mode toggles

`src/InputManager.js` — add M and G keys

Only the changed sections shown — add the highlighted lines:

InputManager with moon gravity and god mode key detection added
InputManager with moon gravity and god mode key detection added

`src/world/PlayerController.js` — add god mode to damage

Only one small change — in `damageFromX`, skip damage when god mode is on:

Created a file, read a file
Created a file, read a file

Here's a summary of every change and what you need to do in **one more file** you'll edit yourself.

Changes made

`DebugOverlay.js` — full rewrite with a two-column HUD panel showing FPS, frame, level ID, position, velocity, HP, score, leaves, enemies, dead/won state, and a live event log. Adds `toggleMoonGravity()` (sets `world.gravity.y` to 1.6 or 10) and `toggleGodMode()`.

`InputManager.js` — added `M` and `G` keys as edge-triggered presses (`moonGravityPressed`, `godModePressed`).

`PlayerController.js` — accepts `debugOverlay` in its options, and `damageFromX()` now returns early if `debugOverlay.godMode` is true.

	### One manual edit needed in `Game.js`
	
	You need to wire the new keys and pass `debugOverlay` to `PlayerController`. In your `Game.js`, find where input is read each frame (likely in `update()`) and add:
	
	js
	// After existing toggle checks (debugTogglePressed → debugOverlay.toggle()):
	if (inputSnap.moonGravityPressed)
	this.debugOverlay?.toggleMoonGravity();
	if (inputSnap.godModePressed)
	this.debugOverlay?.toggleGodMode();
	
	And wherever `PlayerController` is constructed (likely in `Level.js`), pass in `debugOverlay`:
	
	js
	new PlayerController(playerEntity, { events, debugOverlay: this.debugOverlay })
	
	Upload `Game.js` and `Level.js` if you want me to make those edits for you!

The keys don't work at all, as they are not doing their intended functions, and they're also not displayed on the toggle T interface.	You're right — the issue is that the new keys (`M`, `G`) are detected in `InputManager` but never actually *called* in `Game.js`. I need to see that file to fix it properly. Can you upload **`src/Game.js`**?
[upload Game.js]	Found the issue. The `update()` method in `Game.js` only handles `debugTogglePressed` — it never calls `toggleMoonGravity()` or `toggleGodMode()`. I just need to add those two lines. Created a file, read a file Created a file, read a file That was the missing piece. The problem was `Game.js` never called `toggleMoonGravity()` or `toggleGodMode()` — the keys were being detected by `InputManager` but the signals were just ignored.

	What changed in `Game.js`: • Added `moonGravityPressed` and `godModePressed` to the fallback input block (for safety) • Added two new `if` checks right after the existing `debugTogglePressed` block that call `this.debug.toggleMoonGravity()` and `this.debug.toggleGodMode()` **Your final key layout:** • `T` — toggle debug overlay (unchanged) • `M` — toggle moon gravity • `G` — toggle god mode [Game.js changes file]
I need the moon gravity to dissapear when I revive, and also the god mode still doenst work	I need to see how the player takes damage. Can you upload **`src/world/PlayerEntity.js`** and **`src/Level.js`**?
[upload files]	Now I can see both issues clearly: 1. **Moon gravity** — `world.gravity.y` gets set in `DebugOverlay.toggleMoonGravity()` but `Level.restart()` calls `world.gravity.y = this.GRAVITY` only in `build()`, not in `restart()`. So moon gravity persists after restart. 2. **God mode doesn't work** — `PlayerController` receives `debugOverlay` via `opts.debugOverlay`, but in `Level.js` line where `PlayerController` is constructed, it only passes `{ events: this.events }` — `debugOverlay` is never passed in, so it's always `null`. Here are the two small fixes: Level.js with gravity reset on restart and debugOverlay passed to PlayerController Level.js with gravity reset on restart and debugOverlay passed to PlayerController Now `Game.js` also needs to pass `debugOverlay` when constructing `Level`. It currently passes `hudGfx` and `events` but not `debugOverlay`: Edited a file, read a file

	Edited a file, read a file Two root causes fixed: **Moon gravity persisting on restart** — `Level.restart()` now calls `world.gravity.y = this.GRAVITY` to reset physics, and also sets `debugOverlay.moonGravity = false` so the HUD indicator matches reality. **God mode not working** — `debugOverlay` was never making it to `PlayerController`. The chain was broken in two places: `Game.build()` wasn't passing it to `Level`, and `Level.build()` wasn't passing it to `PlayerController`. Both are now fixed — `Game` → `Level` → `PlayerController` all pass it through correctly.